Algorytm wektorów wspierających (SVM)

Celem ćwiczenia jest zbadanie zdolności uogólniania klasyfikatora SVM oraz wpływu funkcji jądra i parametrów modelu na wyniki klasyfikacji. Do ćwiczenia należy opracować własny kod oraz uzasadnić przyjęte rozwiązania.

Środowisko i biblioteki

Środowisko implementacji może być dowolne. W języku Python:

- `numpy` i `pandas`: przygotowanie danych i zestawienie wyników;
- `scikit-learn`: klasyfikator `SVC`, przetwarzanie danych, walidacja krzyżowa i miary oceny;
- `matplotlib`: wykresy wyników.

Wymagane biblioteki są zainstalowane na indywidualnych kontach.

Zakres badań

Zdolność uogólniania należy wyznaczyć za pomocą k-krotnej walidacji krzyżowej, powtórzonej n razy, gdzie k oznacza liczbę części (foldów), a n liczbę powtórzeń całej procedury. W każdym powtórzeniu należy zastosować nowy losowy podział na k części, przy czym wszystkie porównywane konfiguracje muszą być oceniane na dokładnie tych samych podziałach danych. Dla każdego podziału i każdej konfiguracji należy utworzyć nowy klasyfikator SVM (o parametrach danej konfiguracji, uczony od początku na odpowiedniej części uczącej). Wartości k i n trzeba ustalić przed rozpoczęciem badań oraz podać w raporcie. Podstawowym kryterium porównawczym jest dokładność. Warto również analizować liczbę wektorów wspierających.

Badanie dotyczy klasyfikacji binarnej lub wieloklasowej, w zależności od wykorzystywanego zbioru danych. Należy przeprowadzić:

1. porównanie jądra wielomianowego i gaussowskiego (`rbf`) przy ustalonych parametrach obu modeli;
2. badanie wpływu C przy ustalonych pozostałych parametrach;
3. badanie wpływu γ dla jądra gaussowskiego przy ustalonym C.

Dodatkowo można zbadać siatkę kombinacji (C, γ) dla jądra gaussowskiego. W każdym porównaniu należy jasno wskazać parametry zmieniane i parametry utrzymywane na stałym poziomie. W przypadku jądra wielomianowego trzeba podać również wartości jego stopnia (`degree`) oraz parametrów `gamma` i `coef0`. Wartości analizowanych parametrów należy ustalić przed oceną porównywanych wariantów.

Tabela 1: Odpowiedniki funkcji jądra w środowisku MATLAB i języku Python

Funkcja jądra	**MATLAB:** `fitcsvm`	**Python:** `SVC`
liniowa	`linear`	`linear`
wielomianowa	`polynomial`	`poly`
gaussowska	`gaussian`	`rbf`

Uwaga 1: Przy realizacji eksperymentów w obu środowiskach: zgodność nazw jąder nie oznacza zgodności wartości ich parametrów. W MATLAB-ie parametr C to `BoxConstraint`, a stopień wielomianu to `PolynomialOrder`; w SVC są to odpowiednio `C` oraz `degree`. Dla jądra gaussowskiego MATLAB używa `KernelScale` $= s$, a SVC używa `gamma` $= \gamma$. Przy takim samym skalowaniu danych te same funkcje jądra otrzymamy dla $\gamma = 1/s^2$. Dla jądra wielomianowego trzeba ponadto ustawić w obiekcie SVC parametr `coef0` $= 1$.

Uwaga 2: Dla jądra gaussowskiego w `SVC` zachodzi $\gamma = 1/(2\sigma^2)$, gdzie σ oznacza parametr we wzorze funkcji jądra.

Organizacja podziałów i zapobieganie wyciekowi danych

Indeksy wszystkich $n \times k$ podziałów należy wygenerować jeden raz. Dla danego powtórzenia i foldu wszystkie porównywane konfiguracje muszą otrzymać dokładnie te same rekordy uczące i walidacyjne. Dla każdej konfiguracji i każdego podziału należy utworzyć nowy model.

Przykładowe przygotowanie podziałów w języku Python przedstawiono poniżej:

```
from sklearn.model_selection import RepeatedStratifiedKFold
        k, n = 5, 10
        cv = RepeatedStratifiedKFold(
        n_splits=k,
        n_repeats=n,
        random_state=42
)
splits = list(cv.split(X, y))
```

Przed rozpoczęciem badań należy określić, czy standaryzacja danych jest potrzebna, oraz uzasadnić przyjętą decyzję.

Dla każdego podziału należy:

1. wydzielić nieprzetworzone dane uczące i walidacyjne;

2. jeżeli jest to potrzebne, dopasować standaryzację oraz inne operacje zależne od danych, np. uzupełnianie braków lub kodowanie cech, wyłącznie na części uczącej;
3. przekształcić część uczącą i walidacyjną przy użyciu parametrów wyznaczonych na danych uczących;
4. utworzyć i nauczyć oddzielny model dla każdej porównywanej konfiguracji, a następnie ocenić go na tej samej części walidacyjnej.

Jeżeli standaryzacja jest stosowana, jej parametrów nie wolno wyznaczać na całym zbiorze przed walidacją krzyżową. W każdym foldzie należy utworzyć nowy scaler, dopasować go wyłącznie do części uczącej, a następnie zastosować do części uczącej i walidacyjnej:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_val_scaled = scaler.transform(X_val)
```

Tak przygotowany podział można wykorzystać we wszystkich konfiguracjach wymagających tego samego sposobu przetwarzania.

Trenowanie klasyfikatora SVM

Do realizacji procesu uczenia modelu SVM w języku Python można wykorzystać klasę `SVC`. Poniższy fragment pokazuje uczenie i ocenę pojedynczej konfiguracji na jednym podziale danych:

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
model = SVC(
        C=1.0,
        kernel="rbf",
        gamma=0.2,
        cache_size=200
        )
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_val_scaled)
accuracy = accuracy_score(y_val, y_pred)
n_sv = len(model.support_)
```

Liczba `n_sv` dotyczy modelu nauczonego na danym foldzie. Parametr `cache_size` określa rozmiar pamięci podręcznej używanej podczas obliczania jądra, w MB. Dla każdej konfiguracji należy zapisać wyniki uzyskane w kolejnych foldach i powtórzeniach; nie ma potrzeby przechowywania wszystkich nauczonych modeli.

Prezentacja wyników

Wyniki należy zaprezentować w następujący sposób:

- wykresy zmian średniej wartości $\text{Accuracy}_{\text{CV}}$ oraz jej odchylenia standardowego dla porównywanych konfiguracji SVM. Przykładowe zakresy dla jądra gaussowskiego:
  - $C \in \{0.1, 1, 10, 100, 1000, 10000, 100000\}$,
  - $\gamma \in \{0.01, 0.05, 0.2, 1, 2.5, 5, 10, 50, 100\}$.

  W jednym powtórzeniu $\text{Accuracy}_{\text{CV}}$ stanowi średnią dokładność z k foldów. Średnią i odchylenie standardowe wyznacza się następnie z n takich wartości. W ramach jednego eksperymentu porównywane warianty należy umieścić na wspólnym wykresie. Podane zakresy parametrów są przykładami; zakres faktycznie wykorzystany w badaniu należy wskazać w raporcie;
- opcjonalnie, przy badaniu siatki (C, γ), na jednym wykresie można przedstawić $\text{Accuracy}_{\text{CV}} = f(\gamma)$ dla kilku wartości C;
- boxploty wartości $\text{Accuracy}_{\text{CV}}$ uzyskanych dla porównywanych konfiguracji; każdy wariant jest reprezentowany przez n wartości;
- zależność liczby wektorów wspierających od analizowanych konfiguracji. W jednym powtórzeniu należy najpierw uśrednić ich liczbę z k foldów, a następnie zestawić wyniki z n powtórzeń.

Do wygenerowanych wykresów należy dodać odpowiednie wnioski. Wyniki powinny być również zestawione w tabeli podsumowującej.

Problemy do uwzględnienia

- rozpoznanie cech ilościowych i kategorycznych oraz sposób ich kodowania;
- wpływ skalowania cech na wynik klasyfikacji;
- dobór jądra oraz parametrów C, γ i - dla jądra wielomianowego - stopnia wielomianu;
- związek liczby wektorów wspierających z badaną konfiguracją modelu.

Opcjonalnie można przeprowadzić analizę czasu uczenia, przedstawić wyniki dla dodatkowych metryk oraz dokonać interpretacji macierzy pomyłek.

Raport z zadania

W raporcie należy opisać:

- przygotowanie danych: liczbę rekordów, liczbę cech i klas oraz rozkład klas wyrażony liczbą rekordów w każdej klasie;
- podstawy teoretyczne modelu, ze szczególnym uwzględnieniem elementów analizowanych w badaniach;
- protokół oceny, wartości analizowanych parametrów i sposób porównania konfiguracji;
- kryterium wyboru modelu.

Poprawność rozwiązania wymaga, aby wszystkie porównywane konfiguracje były oceniane na tych samych podziałach danych.

Dodatkowe zadanie

Jeżeli wynik najlepszego modelu ma być interpretowany jako końcowa ocena zdolności predykcyjnej, należy użyć niezależnego zbioru testowego. Wymaga to wcześniejszego podziału danych z zachowaniem proporcji klas na zbiór uczący, zawierający na przykład 80% lub 90% rekordów, oraz zbiór testowy, zawierający odpowiednio 20% lub 10% rekordów. Walidację krzyżową i wybór konfiguracji należy przeprowadzić wyłącznie na zbiorze uczącym.

Po wyborze konfiguracji wszystkie operacje przetwarzania danych należy ponownie dopasować na całym zbiorze uczącym, a następnie zastosować do zbioru uczącego i testowego. Model należy ponownie nauczyć na całym zbiorze uczącym i tylko raz ocenić na niezależnym zbiorze testowym.

W przeciwnym przypadku można pozostać przy powtarzanej walidacji krzyżowej. Należy jednak pamiętać, że jest to ocena porównawcza konfiguracji, a nie niezależny test końcowy.